CHAPTER 27

Library Reference: Enumerators and Containers

In this chapter, we'll take a closer look at Ruby's collection classes, especially those features that are based on the Enumerable module, which is the basis for the functionality of all container classes in Ruby. The goal is to give you more information about what you can do with these classes, and also to discuss related functions together so that you can browse and perhaps find a new feature that might help. However, this is not intended to be a complete listing of every class, every method, or every option. For that, please refer to the official Ruby documentation at https://docs.ruby-lang.org.

Array

Arrays are ordered, integer-indexed collections of any object. They are created using the literal square bracket syntax discussed in Chapter 4, Collections, Blocks, and Iterators, on page 53. The %w delimiter can create an array of strings, and %i can create an array of symbols. The method, Array.new(size, default = nil) creates a new array with the given size and populated with the default object. The Kernel#Array method converts its argument into an array if the argument is not already an array.

Arrays implement each, so all methods of Enumerable described in Enumerable, on page 535, can be applied to arrays. In this section, we'll focus on the things Array does above and beyond what it includes from Enumerable.

Accessing Array Values

Arrays use square brackets for access, and the square brackets can contain one of three things:

- A single integer, as in array[3]. Zero returns the first element of the array and a positive integer counts forward, so array[1] is the second element. A negative index counts from the end, with -1 being the last element, -2 the next to last, and so on. If the index is outside the actual array, it returns nil.
- Two integers separated by a comma, as in array[2, 3]. In this case you get the subarray starting at the index represented by the first integer and the length indicated by the second integer. The first integer is interpreted the way it would be if it was passed by itself, meaning that negative numbers count from the end of the array. If the length goes